

Name:Manav Mangesh Uttekar

Roll no.: 71

Problem statement: Beginning with an empty binary search tree, Construct binary search tree by inserting the values in the order given. After constructing a binary tree -

- i. Insert new node**
- ii. Find number of nodes in longest path from root**
- iii. Minimum data value found in the tree,**
- iv. Change a tree so that the roles of the left and right pointers are swapped at every node**
- v. Search a value**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# 1. Insert into BST
def insert(root, key):
    if not root:
        return Node(key)
    if key < root.data:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

# 2. Height of tree (longest path)
def longest_path(root):
    if not root:
        return 0
    return 1 + max(longest_path(root.left), longest_path(root.right))

# 3. Minimum value in BST
def find_min(root):
    current = root
    while current.left:
        current = current.left
    return current.data

# 4. Mirror the tree
def mirror(root):
    if root:
        root.left, root.right = mirror(root.right), mirror(root.left)
    return root

# 5. Search in BST
def search(root, key):
    if not root:
        return False
    if root.data == key:
        return True
```

```

        return search(root.left, key) if key < root.data else
search(root.right, key)

# Inorder Traversal for display
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=' ')
        inorder(root.right)

# --- MAIN ---

# Starting with an empty BST
values = [50, 30, 70, 20, 40, 60, 80]
root = None
for val in values:
    root = insert(root, val)

print("Inorder traversal of BST:")
inorder(root)

# Insert a new node
root = insert(root, 65)
print("\n\nAfter inserting 65:")
inorder(root)

# Longest path from root
print("\n\nLongest path (height):", longest_path(root))

# Minimum data value
print("Minimum value in BST:", find_min(root))

# Mirror the tree
root = mirror(root)
print("Inorder after mirroring:")
inorder(root)

# Search for a value
key = 60
print(f"\n\nSearch {key}:", "Found" if search(root, key) else "Not
found")

```

OUTPUT:

```
python -u C:\Users\manav\Desktop\exp of DSA\exp3.py  
Inorder traversal of BST:  
20 30 40 50 60 70 80  
  
After inserting 65:  
20 30 40 50 60 65 70 80  
  
Longest path (height): 4  
Minimum value in BST: 20  
Inorder after mirroring:  
80 70 65 60 50 40 30 20  
  
Search 60: Not found
```